Project 2

CS6.401 Software Engineering

Submitted By- TEAM 20

Roll Number	Name
2025204002	Sanapathi Kishore Naidu
2025204012	Vadali SS Bharadwaja
2025204018	K. Anil Kumar
2025204033	Deepti Koranga
2025204037	Rahul Ebenezer Chand

Contents

1 Task 1A: Stars Feature Implementation **5**

1.1 Problem Statement 5

1.2 Design Overview 5

1.3 Design Patterns Used 6

1.3.1 Model–View–Controller (MVC) 6

1.3.2 Service Abstraction Pattern 6

1.3.3 Data Access / Repository Query Encapsulation 6

1.4 Implementation Details 7

1.4.1 Key Components Added / Modified 7

1.4.2 UI Behavior 8

1.4.3 Database Schema Evolution 8

1.4.4 Toggle Mechanism 8

2 Task 1B: Trending Blogs **9**

2.1 Feature Overview 9

2.2 Functional Requirements 9

2.3 System Design and Architecture 10

2.4 Efficient Data Retrieval Strategy 10

2.5 Design Patterns Used 11

2.6 Implementation Details 11

3 Task 2: Transforming Feeds **12**

3.1 Overview 12

3.2 Implementation 13

3.2.1 Content Processing Pipeline 13

3.3 Processors Implemented 14

3.3.1 Profanity Filter Processor 14

3.3.2 Summary Processor 14

3.3.3 Tag Generator Processor 14

3.4 API Integration 15

3.5 Integration with Apache Roller 15

3.6 Design Principles Used 15

3.6.1 Chain of Responsibility Pattern 15

3.6.2 Single Responsibility Principle (SRP) 16

3.6.3 Open/Closed Principle 16

3.6.4 Modularity and Extensibility 16
3.7 Trade-offs 17

4 Task 3: Manage Issues Feature 17
4.1 Feature Overview 17
4.1.1 Capability Matrix 18
4.2 Database Schema Design 18
4.3 Architecture: Strategy Pattern 18
4.3.1 Why Strategy Pattern 19
4.3.2 Structure 19
4.3.3 Instantiation Points 19
4.3.4 Method Signature Split 19
4.4 SOLID Principles Applied 19
4.4.1 Single Responsibility Principle 20
4.4.2 Open/Closed Principle 20
4.4.3 Liskov Substitution Principle 20
4.4.4 Interface Segregation Principle 20
4.4.5 Dependency Inversion Principle 20
4.5 Technology Choices and Rationale 21
4.5.1 Email Service Integration 21
4.5.2 Configuration and Credential Management 21
4.6 Complete User & Admin Flows 21
4.6.1 User Submits Issue 21
4.6.2 User Updates Issue 22
4.6.3 Admin Resolves Issue 22
4.6.4 Admin Deletes Issue 22
4.7 Comparative Analysis & Trade-offs at Scale 22
4.7.1 Query Scaling Behaviour 22
4.7.2 Eager Loading vs Pagination 22
4.7.3 ORM Configuration Decisions 22
4.7.4 Email Delivery Strategy 22
4.7.5 Admin Lookup Strategy 23

5 Task 4: Admin Dashboard (Site Summary Dashboard) 23
5.1 Introduction 23
5.2 Objectives 23
5.3 Dashboard Features 23

5.4 Dashboard Views . . . 24
5.4.1 Minimalist View . . . 24
5.4.2 Full View . . . 25
5.5 System Architecture and Design . . . 25
5.6 Design Patterns Used . . . 25
5.6.1 Builder Pattern . . . 25
5.6.2 Model–View–Controller (MVC) . . . 26
5.6.3 DAO / Repository Pattern . . . 26
5.6.4 Singleton / Service Locator . . . 26
5.6.5 Data Transfer Object (DTO) . . . 27
5.7 Advantages of Implementation . . . 27
5.8 Conclusion . . . 27

6 Task 5A: Web Translation Feature 27
6.1 Feature Overview . . . 27
6.2 Supported Languages . . . 28
6.3 Translation Workflow . . . 28
6.4 Translation Providers . . . 29
6.5 System Architecture . . . 29
6.6 Design Patterns Used . . . 30
6.7 Scalability and Performance Considerations . . . 30
6.8 User Interface Demonstration . . . 31
6.9 Future Improvements . . . 31

7 Task 5B: Translation Caching Feature 32
7.1 Feature Overview & Task Flow . . . 32
7.2 Design Principles Integrated . . . 32
7.3 Leveraged Design Patterns . . . 33
7.4 Quality Attributes & Trade-offs . . . 34

8 Task 6A: Community Pulse Dashboard 35
8.1 Introduction . . . 35
8.2 Overview of Community Pulse Dashboard . . . 35
8.3 Implementation Details . . . 36
8.4 Design Patterns Used . . . 38
8.5 Conclusion . . . 39

9 Task 6B: Community Pulse Conversation Breakdown 40

9.1 Overview of Task . . . 40
9.2 SOLID Design Principles Applied . . . 40
9.2.1 Single Responsibility Principle . . . 40
9.2.2 Open/Closed Principle . . . 40
9.2.3 Liskov Substitution Principle . . . 41
9.2.4 Interface Segregation Principle . . . 41
9.2.5 Dependency Inversion Principle . . . 41
9.3 Design Patterns Used . . . 41
9.3.1 Strategy Pattern . . . 42
9.3.2 Factory Pattern . . . 42
9.3.3 Template Pipeline Approach . . . 42
9.4 Implementation Architecture . . . 42
9.4.1 Core Files Added . . . 43
9.4.2 Existing Files Updated . . . 43
9.5 Method Behaviors . . . 44
9.5.1 Classical NLP . . . 44
9.5.2 LLM-Assisted . . . 44
9.5.3 Hybrid Approach . . . 44
9.6 Auto-Selection Logic . . . 45
9.7 UI and Interaction . . . 45

10 Team Member Contributions **45**

1 Task 1A: Stars Feature Implementation

1.1 Problem Statement

Modern blogging platforms often allow users to bookmark or mark content they find useful so that it can be easily accessed later. However, the default Apache Roller platform does not provide a built-in mechanism for users to mark blogs or blog entries as favourites.

The **Stars Feature** was introduced to address this limitation. The feature allows authenticated users to mark weblogs or individual blog posts as "starred," enabling quick access to preferred content. This functionality enhances the user experience by helping users track blogs and posts they frequently visit.

Two types of starring are supported:

- **Starring a weblog** – users can mark an entire blog as a favourite.
- **Starring a blog entry** – users can mark individual posts for later viewing.

Starred weblogs appear on the homepage and are sorted based on the most recent blog post date. Starred blog entries can be accessed through a dedicated page with pagination support.

1.2 Design Overview

The Stars feature is implemented as a modular extension to the Apache Roller architecture. The design follows a layered structure consisting of domain entities, service interfaces, persistence logic, and web actions.

Two new domain entities were introduced:

- `StarredWeblog` – represents a relationship between a user and a starred weblog.
- `StarredEntry` – represents a relationship between a user and a starred blog entry.

These entities store information about which users have marked specific blogs or entries as favourites.

The architecture is organized into the following layers:

- **Application Layer** – provides service interfaces that allow users to perform star-related operations such as starring, unstarring, and retrieving starred items.
- **Persistence Layer** – stores star relationships in the database and manages data access operations.
- **Web Layer** – exposes the functionality through Struts2 actions, enabling users to interact with the feature through the web interface.

The design ensures that the feature integrates seamlessly with the existing Apache Roller architecture.

1.3 Design Patterns Used

Several design patterns were applied to ensure maintainability, modularity, and separation of concerns in the Stars feature implementation.

1.3.1 Model–View–Controller (MVC)

The implementation follows the MVC architecture, which separates application logic into three layers.

- **Model**: Entities such as `StarredWeblog` and `StarredEntry` represent the data structure.
- **Controller**: Struts2 actions such as `StarAction` and `StarredEntriesAction` handle user requests and coordinate between services and views.
- **View**: JSP pages display starred weblogs and entries to the user.

This separation ensures that changes in the user interface do not affect business logic or data management.

1.3.2 Service Abstraction Pattern

The Service Abstraction Pattern is used to define high-level operations for star-related functionality through an interface called `UserService`.

This interface declares operations such as:

- Starring or unstarring weblogs
- Starring or unstarring entries
- Retrieving starred weblogs
- Retrieving starred entries

The actual implementation of these methods is handled by `JPAUserManagerImpl`, which interacts with the database and manages business logic.

This abstraction allows the service implementation to change without affecting other parts of the system.

1.3.3 Data Access / Repository Query Encapsulation

Database operations related to starring functionality are encapsulated within the persistence layer.

The `JPAUserManagerImpl` class performs operations such as:

- Checking whether a blog entry is already starred

- Inserting new star records
- Deleting star records when users unstar items
- Retrieving lists of starred blogs and entries

Encapsulating database queries in a dedicated manager class improves maintainability and keeps database logic separate from presentation and controller layers.

1.4 Implementation Details

The Stars feature was implemented by introducing new components across multiple layers of the Apache Roller system, including domain entities, services, web actions, and user interface components.

1.4.1 Key Components Added / Modified

Several classes and components were added or modified to implement the feature.

Domain Layer

- `StarredWeblog.java` – represents a starred weblog relationship between a user and a blog.
- `StarredEntry.java` – represents a starred blog entry.

Service Layer

- `UserService.java` – declares star-related operations.
- `JPAUserManagerImpl.java` – implements star logic and manages database interactions.

Observer Components

- `StarObserver.java` – observer interface for star-related events.
- `StarLoggingObserver.java` – logs star/unstar operations.

Web Layer

- `StarAction.java` – handles HTTP requests for starring and unstarring entries.
- `StarredEntriesAction.java` – displays a paginated list of starred entries.
- `MainMenu.java` – shows starred weblogs on the homepage.

View Layer

- JSP files that render starred weblog lists and starred entries pages.

1.4.2 UI Behavior

The user interface provides interactive elements for managing starred content.

Users can:

- Click a star icon to mark a blog or entry as favourite.
- View starred weblogs directly on the homepage.
- Access a separate page listing starred entries.

The starred entries page supports pagination with a default page size of 10 entries, ensuring efficient navigation through large lists.

Additionally, starred weblogs display the timestamp of their most recent blog post to help users identify active blogs.

1.4.3 Database Schema Evolution

To support the stars feature, new tables were introduced in the database to store relationships between users and starred items.

These tables store:

- User identifiers
- Weblog or entry identifiers
- Metadata required for retrieving starred items

By separating these relationships into dedicated tables, the system avoids modifying existing blog or entry tables while still enabling efficient queries for starred content.

1.4.4 Toggle Mechanism

The star interaction mechanism is implemented using a toggle operation.

When a user clicks the star icon:

1. The request is sent to `StarAction`.
2. The controller calls the `toggleStarEntry()` or `toggleStarWeblog()` method in the service layer.
3. The system checks whether the item is already starred.
4. If the item is not starred, a new record is inserted into the database.
5. If the item is already starred, the existing record is removed.

After the operation, observers are notified through the **Observer Pattern**, allowing components such as logging services to react to star or unstar events.

2 Task 1B: Trending Blogs

2.1 Feature Overview

The **Trending Blogs** feature highlights the most popular content on the Apache Roller platform by displaying the top-performing blog posts and weblogs based on user engagement.

Specifically, the system displays:

- The top 5 most-starred blog posts
- The top 5 most-starred weblog pages

This feature helps users quickly identify the most popular content on the platform. It is especially useful for users who want to discover trending topics or avoid missing out on highly engaging blog posts.

The feature improves user engagement by promoting widely appreciated content and helping readers discover blogs that are currently receiving the most attention.

2.2 Functional Requirements

The **Trending Blogs** feature must satisfy several functional requirements to ensure both usability and performance.

The system must:

- Display the top five most-starred blog entries on the platform.
- Display the top five most-starred weblog pages.
- Ensure the trending list is generated efficiently using database-level aggregation queries.
- Avoid iterating through individual posts in Java code for performance reasons.
- Use SQL/JPQL aggregation operations such as:
  - `GROUP BY`
  - `COUNT`
  - `ORDER BY`
- Maintain fast response times even when the number of blog posts grows significantly.

Additionally, the system should ensure that the trending list always contains five items. If fewer than five starred posts exist, the remaining slots are filled with recently published blog entries.

2.3 System Design and Architecture

The architecture of the Trending Blogs feature follows a layered structure that separates responsibilities between business logic, web actions, strategies, and the presentation layer.

The major components involved in this feature include:

Layer	File	Role
Business Interface	`UserManager.java`	Declares methods `getTrendingEntries(int`
Business Implementation	`JPAUserManagerImpl.java`	Executes optimized JPQL aggregate queries
Web Action	`TrendingBlogsAction.java`	Context class that manages the active trendi
Strategy Interface	`TrendingStrategy.java`	Defines method `getTrendingBlogs(int)`
Concrete Strategy	`DefaultTrendingStrategy.java`	Implements the trending algorithm and retri
View Layer	`TrendingBlogs.jsp`	Displays trending blog posts to users

This architecture ensures a clean separation between data retrieval, algorithm selection, and presentation logic.

2.4 Efficient Data Retrieval Strategy

The Trending Blogs feature emphasizes efficient database-level computation to ensure scalability and performance.

Instead of retrieving all blog posts and processing them in application code, the system performs aggregation directly within the database using optimized JPQL queries.

The main query used for retrieving trending entries is:

```
SELECT se.entry.id, COUNT(se) AS starCount
FROM StarredEntry se
GROUP BY se.entry.id
ORDER BY starCount DESC
```

Key aspects of this approach include:

- Aggregation operations such as `COUNT` and `GROUP BY` are executed at the database level.
- The query returns only the top entries with the highest star counts.
- This approach avoids inefficient iteration through all blog entries in Java.
- Database engines are optimized for aggregation operations, resulting in better performance.

If fewer than five entries have stars, the system retrieves additional entries using a secondary query that selects recently published blog posts. This padding logic ensures that the trending list always contains five items.

2.5 Design Patterns Used

The implementation of the Trending Blogs feature uses the **Strategy Pattern** to encapsulate the ranking algorithm used to determine trending content.

The Strategy Pattern is a behavioural design pattern that defines a family of algorithms and allows them to be selected dynamically at runtime.

In this implementation:

- The interface `TrendingStrategy` defines the contract for retrieving trending blogs.
- `DefaultTrendingStrategy` implements the algorithm that ranks blogs based on star counts.
- `TrendingBlogsAction` acts as the context that holds and invokes the strategy.

This design provides several advantages:

- The definition of "trending" can evolve over time without modifying the main application logic.
- Different ranking algorithms can be introduced, such as:
  - Weighted recency scoring
  - Comment-based popularity
  - Machine learning–based ranking
- Strategies can be independently tested and maintained.
- The web action and view layers remain decoupled from the ranking algorithm.

Roles in the Strategy Pattern:

GoF Role	Class
Strategy	`TrendingStrategy`
Concrete Strategy	`DefaultTrendingStrategy`
Context	`TrendingBlogsAction`

2.6 Implementation Details

The implementation of the Trending Blogs feature involves several components that work together to retrieve, process, and display trending content.

The workflow of the feature is as follows:

1. A user accesses the trending blogs page through the browser using a URL such as:

```
GET /roller-ui/trendingBlogs.rol
```

2. The request is received by `TrendingBlogsAction`.

3. The action invokes the active strategy by calling:

```
DefaultTrendingStrategy.getTrendingBlogs(5)
```

4. The strategy retrieves trending entries using the business layer method:

```
JPAUserManagerImpl.getTrendingEntries(5)
```

5. The business layer executes the optimized JPQL aggregation query in the database.

6. The database returns a list of `(entryId, starCount)` pairs.

7. The strategy retrieves the full blog entry objects using:

```
WeblogEntryManager.getWeblogEntry(id)
```

8. The retrieved blog entries are converted into `TrendingPost` objects, which contain fields such as:

   - `id`
   - `title`
   - `numberOfStars`
   - `permalink`

9. These objects are returned to `TrendingBlogsAction`, which forwards them to the view layer.

10. Finally, `TrendingBlogs.jsp` renders the trending posts on the web page.

This process ensures that trending blog content is generated efficiently and displayed dynamically to users.

3 Task 2: Transforming Feeds

3.1 Overview

The Transforming Feeds feature introduces an AI-powered content processing pipeline integrated into the Apache Roller Weblogger system. The goal of this system is to automatically enhance blog post content before it is stored or published. Instead of manually editing posts, the pipeline processes the content through several automated stages that improve readability, safety, and metadata quality. The pipeline performs multiple transformations including profanity filtering, content summarization, tag generation, and mood analysis. Each transformation is implemented as an independent processor in a modular architecture. This allows content to be progressively enhanced while maintaining flexibility and extensibility. The system leverages Groq's LLM API

for intelligent content processing while following the Chain of Responsibility design pattern to organize the processing stages. This design ensures that each processor performs a specific task and then passes the modified content to the next stage in the pipeline.

3.2 Implementation

The implementation is based on a modular processing pipeline that integrates with the Apache Roller blog entry saving workflow. Whenever a user saves a blog post, the content is passed through the pipeline before being stored in the database.

The processing architecture consists of:

- A handler interface that defines the processing contract
- A base processor class that implements the chaining logic
- Multiple concrete processors that perform specific transformations
- A pipeline builder that constructs and executes the processing chain
- A hook integration point that connects the pipeline with Apache Roller

This modular structure ensures that each transformation step operates independently while still contributing to the final enhanced content.

3.2.1 Content Processing Pipeline

The Content Processing Pipeline is responsible for executing the sequence of processors that transform blog content.

The processing flow is as follows:

1. The user writes and saves a blog post.
2. Apache Roller triggers the `BlogEntryProcessorHook` during the save operation.
3. The hook calls the `ContentProcessingPipeline`.
4. The pipeline executes a sequence of processors that transform the content.
5. The processed content is returned and stored in the database.

According to the architecture diagram on page 2 of the document, the data flow is:

`User Saves Blog Post` → `BlogEntryProcessorHook` → `ContentProcessingPipeline` → `Processor Chain` → `Enhanced Content` → `Database`

Each processor modifies the content and passes the result to the next processor in the chain.

3.3 Processors Implemented

The pipeline consists of multiple processors, each responsible for performing a specific transformation on the blog content.

3.3.1 Profanity Filter Processor

The `ProfanityProcessor` is responsible for sanitizing blog content by detecting offensive or inappropriate words and replacing them with safer alternatives.

This processor ensures that published content remains professional and suitable for public platforms. Instead of removing content entirely, the processor replaces offensive expressions with more acceptable language.

For example:

- **Input:**

  "This damn system is shit..."

- **Output:**

  "This complicated system is a mess..."

By performing this transformation automatically, the system helps maintain content quality without requiring manual moderation.

3.3.2 Summary Processor

The `SummaryProcessor` automatically generates a short summary of the blog content using LLM-based analysis.

The generated summary typically consists of approximately 40–50 words and is placed at the beginning of the blog post. This allows readers to quickly understand the main topic of the article.

The resulting content format becomes:

Quick Summary: {generated summary}

{original blog content}

This improves readability and enhances user experience by providing quick insights into the blog content.

3.3.3 Tag Generator Processor

The `TagGeneratorProcessor` analyzes the blog content and automatically generates relevant hashtags that represent the key topics discussed in the article.

Typically, the processor generates 3–5 relevant tags and appends them to the end of the blog post.

Example format:

> **Tags:**
>
> #system #technical #challenges

These tags help categorize the blog content and improve searchability within the blogging platform.

3.4 API Integration

The system integrates with Groq's Large Language Model API to perform advanced natural language processing tasks such as summarization, tag generation, and mood analysis. The communication with the AI service is handled through a dedicated component called LLMClient. This client sends blog content to the Groq API and receives processed responses that are used by the pipeline processors. The LLM enables intelligent analysis of blog content and allows the system to generate summaries, detect tone, and identify relevant topics automatically. To maintain security and flexibility, the API key is provided through the website interface rather than being hardcoded into the application configuration.

3.5 Integration with Apache Roller

The pipeline integrates with Apache Roller through the JPAWeblogEntryManagerImpl class. When a blog entry is saved, the system triggers the BlogEntryProcessorHook, which invokes the content processing pipeline. This integration ensures that the pipeline operates transparently within the existing blog publishing workflow. Users do not need to manually trigger the processing; the system automatically enhances the content during the save operation. The integration approach maintains compatibility with the existing architecture of Apache Roller while adding advanced content processing capabilities.

3.6 Design Principles Used

The architecture of the content processing pipeline follows several important software design principles.

3.6.1 Chain of Responsibility Pattern

The pipeline uses the **Chain of Responsibility** design pattern to organize content processors into a sequential chain.

Each processor performs its specific transformation and then passes the modified content to the next processor in the chain.

The main components involved are:

- `ContentProcessor` – defines the processing interface
- `AbstractContentProcessor` – implements chaining behavior
- `Concrete Processors` – perform specific transformations
- `ContentProcessingPipeline` – builds and executes the chain

This pattern enables flexible processing flows and allows processors to be added or removed easily.

3.6.2 Single Responsibility Principle (SRP)

Each processor in the pipeline performs only one specific task.

For example:

- `ProfanityProcessor` → content sanitization
- `SummaryProcessor` → summarization
- `TagGeneratorProcessor` → hashtag generation
- `MoodAnalyzerProcessor` → tone analysis

By isolating these responsibilities, the system becomes easier to maintain, test, and extend.

3.6.3 Open/Closed Principle

The system is designed to be open for extension but closed for modification. New processors can be added to the pipeline without modifying existing processors or the core pipeline logic. Developers simply implement the ContentProcessor interface and add the processor to the pipeline configuration. This allows the pipeline to evolve without disrupting existing functionality.

3.6.4 Modularity and Extensibility

The pipeline architecture emphasizes modular design.

Each processor is implemented as an independent component, allowing developers to:

- Enable or disable processors through configuration
- Add new processors easily
- Modify individual processors without affecting others

This modular approach also supports runtime configuration through the `ProcessorConfig` class.

3.7 Trade-offs

Although the content processing pipeline provides significant benefits, it also introduces several trade-offs.

1. **Performance Overhead**

   Since the pipeline uses multiple processors and external API calls, the total processing time can increase. According to the performance analysis, the full pipeline execution may take approximately 2–8 seconds, depending on API response time.

2. **External API Dependency**

   The system relies on external LLM services. If the AI service becomes unavailable, certain transformations may fail.

   To address this issue, the pipeline includes fault-tolerant mechanisms such as:

   - Skipping the pipeline if no API key is provided
   - Returning the original content if the API response fails
   - Continuing execution even if one processor fails

3. **Content Privacy Concerns**

   Sending blog content to external AI services may raise privacy considerations. Therefore, proper security practices and API key management must be followed.

Despite these trade-offs, the pipeline significantly improves the content publishing workflow by providing automated content enhancement and intelligent analysis.

4 Task 3: Manage Issues Feature

4.1 Feature Overview

The Manage Issues Feature introduces a bug reporting and issue management system within Apache Roller. This feature allows authenticated users to report bugs encountered within the platform, while administrators can manage, resolve, and delete these issues through a dedicated dashboard. The system provides a structured workflow for handling reported issues, ensuring that user feedback is properly tracked and addressed. Each bug report contains information such as title, description, severity level, reporter details, timestamps, and status. Administrators can review these reports and update their status as they investigate and resolve issues. Additionally, the system supports multi-channel email notifications to keep administrators and reporters informed about bug lifecycle events such as creation, resolution, and deletion. Notifications are sent through both a traditional SMTP email channel and an external MailerSend API integration. The feature follows Apache Roller's architecture conventions, including XML-based JPA mapping, Struts2 actions, Guice dependency injection, and Tiles-based UI layouts.

4.1.1 Capability Matrix

The feature supports different capabilities for users and administrators.

Capability	**User**	**Admin**
Submit bug report	✓	✓
View submitted reports	✓ (own reports)	✓ (all reports)
Filter reports by status	✗	✓
Resolve issue	✗	✓
Delete issue	✗	✓
Receive email notifications	✓	✓

This capability matrix ensures that the system enforces proper role-based permissions for issue management.

4.2 Database Schema Design

The bug reporting feature introduces a new database table called `roller_bug_report` to store issue data.

The schema includes the following fields:

- `id` – Unique identifier (UUID) for each report
- `title` – Short description of the issue
- `description` – Detailed explanation of the bug
- `severity` – Priority level (LOW, NORMAL, HIGH, CRITICAL)
- `status` – Current status (OPEN, RESOLVED, DELETED)
- `reporter_id` – Reference to the user who reported the issue
- `created_time` – Timestamp of report creation
- `updated_time` – Timestamp of last modification
- `admin_notes` – Optional notes added by administrators

Indexes are created for fields such as `status`, `reporter_id`, and `created_time` to improve query performance when filtering reports.

The entity is implemented as a plain Java object with mapping defined in `BugReport.orm.xml`, following Apache Roller's XML-based JPA mapping convention.

4.3 Architecture: Strategy Pattern

The notification system for the bug reporting feature is designed using the Strategy Pattern, enabling multiple notification mechanisms to be implemented independently.

4.3.1 Why Strategy Pattern

Different communication channels may be used to notify administrators and users when a bug report is created or updated. Instead of embedding email logic directly into the core bug management logic, the system separates notification behavior into independent strategy implementations. This allows the application to support different notification channels without modifying the core bug reporting system.

4.3.2 Structure

The architecture includes the following components:

- `NotificationChannel` (interface) – defines notification methods
- `EmailNotificationChannel` – sends notifications using SMTP email
- `MailerSendNotificationChannel` – sends notifications through the MailerSend API
- `NotificationServiceImpl` – dispatches events to all registered channels

The manager responsible for bug operations triggers the notification service when events occur.

4.3.3 Instantiation Points

Notification channels are instantiated and registered through Guice dependency injection. The JPAWebloggerModule configuration binds the notification service and registers multiple implementations of the NotificationChannel interface using a multibinder. This allows the system to automatically detect and invoke all available notification channels.

4.3.4 Method Signature Split

The notification interface defines three lifecycle methods corresponding to major bug report events:

- `onBugCreated(report)`
- `onStatusChanged(report, oldStatus)`
- `onBugDeleted(report)`

Each implementation of the `NotificationChannel` interface handles these events independently, ensuring that the same event can trigger multiple notification mechanisms simultaneously.

4.4 SOLID Principles Applied

The implementation follows several SOLID design principles to improve maintainability and extensibility.

4.4.1 Single Responsibility Principle

Each component in the system has a clearly defined responsibility.

For example:

- `BugReportManager` handles bug report persistence and retrieval.
- `NotificationService` manages notification dispatch.
- `NotificationChannel` implementations handle communication through specific channels.

This separation ensures that each component focuses on a single concern.

4.4.2 Open/Closed Principle

The system is designed to be open for extension but closed for modification. New notification channels can be added simply by implementing the NotificationChannel interface and registering the implementation through dependency injection. This approach allows the system to support additional communication channels such as Slack or webhooks without modifying the existing codebase.

4.4.3 Liskov Substitution Principle

All implementations of the `NotificationChannel` interface behave consistently with the base abstraction.

For example:

- `EmailNotificationChannel`
- `MailerSendNotificationChannel`

Both can be substituted interchangeably within the `NotificationService` without affecting the correctness of the system.

4.4.4 Interface Segregation Principle

The NotificationChannel interface contains only the methods necessary for handling bug report lifecycle events. This avoids creating large interfaces that force implementations to support unnecessary functionality.

4.4.5 Dependency Inversion Principle

High-level components such as JPABugReportManagerImpl depend on abstractions rather than concrete implementations. Instead of directly referencing specific notification channels, the manager communicates with the NotificationService interface, which internally manages different channel implementations. This reduces coupling between components and improves system flexibility.

4.5 Technology Choices and Rationale

4.5.1 Email Service Integration

Two different email delivery mechanisms were implemented:

- **SMTP Email** (`EmailNotificationChannel`) – Uses the existing `MailUtil` class to send standard email messages.
- **MailerSend API** (`MailerSendNotificationChannel`) – Integrates with the MailerSend Java SDK for modern email delivery.

Using two independent channels increases reliability. If one channel fails, the other can still deliver notifications.

4.5.2 Configuration and Credential Management

All configuration values are stored in `roller.properties` or `roller-custom.properties`.

Examples include:

- Administrator email address
- MailerSend API token
- Feature enable/disable flags
- Sender email information

Sensitive credentials are stored in configuration files rather than being hardcoded in the application source code.

4.6 Complete User & Admin Flows

The feature supports several workflows involving both users and administrators.

4.6.1 User Submits Issue

1. A user accesses the Bug Report submission form.
2. The user fills in the title, description, and severity level.
3. The form submits a request to `BugReportAction.save()`.
4. The bug report is persisted in the database.
5. `NotificationService` sends notifications to administrators and reporters.

4.6.2 User Updates Issue

Users can view the status of issues they reported. When administrators update the issue status, the reporter receives a notification about the update.

4.6.3 Admin Resolves Issue

1. The administrator opens the Bug Dashboard.
2. The administrator selects a bug report.
3. The status is updated to `RESOLVED`.
4. The change is saved to the database.
5. Notifications are sent to both administrators and the original reporter.

4.6.4 Admin Deletes Issue

If an issue is no longer relevant, the administrator can mark it as DELETED. Instead of permanently removing the record, the system performs a soft delete by updating the issue status. This preserves the audit history while preventing the issue from appearing in normal listings.

4.7 Comparative Analysis & Trade-offs at Scale

4.7.1 Query Scaling Behaviour

Indexes on status, reporter, and creation time fields ensure efficient filtering and retrieval of bug reports even as the number of reports grows.

4.7.2 Eager Loading vs Pagination

To maintain performance when dealing with many bug reports, pagination is used in the admin dashboard instead of loading all records at once. This approach reduces memory usage and improves response time for large datasets.

4.7.3 ORM Configuration Decisions

The system uses XML-based JPA mapping rather than annotation-based entities to maintain consistency with the existing Apache Roller codebase. This decision ensures compatibility with existing persistence infrastructure.

4.7.4 Email Delivery Strategy

Two independent notification channels are implemented:

- **SMTP Email**

- **MailerSend API**

Each channel operates independently, and failures in one channel do not prevent the other from sending notifications.

4.7.5 Admin Lookup Strategy

Administrators receive bug notifications through configuration-defined email addresses. This allows organizations to modify notification recipients without changing the application code.

5 Task 4: Admin Dashboard (Site Summary Dashboard)

5.1 Introduction

The Admin Dashboard (Site Summary Dashboard) provides administrators with a high-level overview of platform activity across the Apache Roller blogging system. Instead of manually querying the database or navigating through multiple sections of the application, administrators can quickly view key engagement metrics from a single interface. The dashboard aggregates important platform statistics such as the number of registered users, blog entries, category popularity, and user activity levels. This information helps administrators monitor system usage, evaluate user engagement, and identify trends within the blogging platform. To accommodate different levels of detail, the dashboard supports multiple viewing modes that allow administrators to switch between simplified and detailed views of the platform metrics.

5.2 Objectives

The primary objectives of the Site Summary Dashboard are:

- Provide administrators with a centralized overview of platform activity.
- Display key engagement metrics in a clear and accessible format.
- Allow administrators to switch between different dashboard views depending on the required level of detail.
- Maintain separation between presentation logic and data retrieval logic.
- Ensure the system is extensible so additional dashboard views and metrics can be added in the future.

By achieving these objectives, the dashboard improves the usability and maintainability of the administrative interface.

5.3 Dashboard Features

The Site Summary Dashboard includes several features designed to present important platform metrics effectively.

The dashboard displays information such as:

- Total number of registered users
- Total number of published blog entries
- Most popular blog category
- Most starred weblog
- Most active users on the platform

These metrics allow administrators to evaluate how users interact with the system and identify the most active contributors.

Additionally, the dashboard supports multiple viewing modes, enabling administrators to control how much information is displayed at any given time.

5.4 Dashboard Views

The dashboard supports two primary view modes: **Minimalist View** and **Full View**. These views allow administrators to toggle between a simplified summary and a more comprehensive analytics dashboard.

Mode	Metrics Shown
Minimalist View	Total users, Top category
Full View	Total users, Total entries, Top category, Most-starred blog, Top-3 active users leaderboar

The logic defining which metrics appear in each view is separated from the data retrieval layer. This ensures that dashboard configurations can be modified without affecting the underlying service layer responsible for fetching data.

5.4.1 Minimalist View

The **Minimalist View** provides a lightweight overview of the system. It displays only the most essential metrics needed for quick monitoring.

The following metrics are included:

- **Total Users** – total number of registered users on the platform.
- **Top Category** – the category containing the highest number of published blog entries.

This view is designed for administrators who require a quick summary without additional details.

5.4.2 Full View

The **Full View** provides a more comprehensive dashboard containing detailed engagement metrics.

Metrics displayed in the Full View include:

Total Users Displays the total number of registered users on the platform.

Total Published Entries Shows the total number of published blog posts across all weblogs.

Top Category Identifies the category that contains the highest number of published entries.

Most Starred Blog Displays the weblog that has received the highest cumulative number of stars from users.

Top 3 Most Active Users Shows a leaderboard of the three users who have published the highest number of blog entries.

This view helps administrators gain deeper insights into user activity and content engagement.

5.5 System Architecture and Design

The architecture of the Site Summary Dashboard follows a layered structure that separates responsibilities between different components of the system.

The key components involved in the dashboard implementation include:

Layer	File	Role
Strategy Interface	`DashboardViewStrategy.java`	Declares the method `getMetricKeys()` wh
Concrete Strategy	`FullDashboardViewStrategy.java`	Returns all metric keys for the full dashbo
Concrete Strategy	`MinimalistDashboardViewStrategy.java`	Returns a subset of metrics for the minim
Context	`DashboardGenerator.java`	Maintains the active strategy and generate
Data Service	`SiteSummaryServiceImpl.java`	Retrieves metric values from the database
Web Action	`SiteSummaryAction.java`	Determines which dashboard strategy shou
View	`SiteSummary.jsp`	Displays the dashboard interface

This architecture ensures that view configuration, business logic, and data access remain independent, making the system easier to maintain and extend.

5.6 Design Patterns Used

Several software design patterns were used in the implementation of the Site Summary Dashboard to improve modularity, flexibility, and maintainability.

5.6.1 Builder Pattern

The Builder Pattern is used in the DashboardGenerator component to construct the dashboard output step-by-step. The generator receives a strategy that specifies which metrics are required

and then builds the final dashboard data structure by retrieving each metric from the data service. This approach allows the dashboard to be assembled dynamically based on the selected view configuration.

5.6.2 Model–View–Controller (MVC)

The dashboard follows the MVC architectural pattern, which separates application responsibilities into three components:

- **Model**: Data provided by `SiteSummaryServiceImpl`.
- **View**: `SiteSummary.jsp` which renders the dashboard interface.
- **Controller**: `SiteSummaryAction` which processes administrator requests and coordinates data generation.

This separation improves maintainability and simplifies future UI changes.

5.6.3 DAO / Repository Pattern

The DAO (Data Access Object) pattern is used in the service layer to encapsulate database operations.

`SiteSummaryServiceImpl` acts as a data-access service responsible for executing queries that retrieve dashboard metrics such as:

- user counts
- entry counts
- category statistics
- star counts
- activity leaderboards

Encapsulating database queries in a dedicated service layer prevents direct database access from controllers or views.

5.6.4 Singleton / Service Locator

The service layer components such as SiteSummaryServiceImpl are managed as shared services within the application. This ensures that a single instance of the service is reused across requests, improving performance and maintaining consistent access to application resources.

5.6.5 Data Transfer Object (DTO)

The dashboard metrics are collected and returned as a Map of key-value pairs, which acts as a lightweight Data Transfer Object (DTO). This DTO transfers metric data from the service layer to the view layer without exposing internal implementation details. Using DTOs helps reduce coupling between system layers and simplifies data communication between components.

5.7 Advantages of Implementation

The implementation of the Site Summary Dashboard provides several advantages:

- **Flexible dashboard configuration** – new dashboard views can be added without modifying existing logic.
- **Separation of concerns** – view logic, data access, and business logic remain independent.
- **Extensibility** – additional metrics can be introduced easily.
- **Improved maintainability** – modular design reduces code complexity.
- **Efficient data retrieval** – targeted database queries are executed only for required metrics.
- **Enhanced administrative insight** – administrators can monitor system engagement effectively.

5.8 Conclusion

The Site Summary Dashboard enhances the administrative capabilities of the Apache Roller platform by providing a centralized interface for monitoring platform activity and engagement metrics. By supporting multiple dashboard views and separating view configuration from data retrieval logic, the system achieves a flexible and extensible architecture. The use of design patterns such as MVC, Builder, DAO, and Strategy further improves maintainability and modularity. Overall, the implementation provides administrators with valuable insights into system usage while maintaining a clean and scalable system architecture.

6 Task 5A: Web Translation Feature

6.1 Feature Overview

The **Web Translation Feature** enables users to translate any weblog page into their preferred Indian language directly within the webpage. The translation occurs in-place, meaning the page content is translated dynamically without reloading the page. This ensures that the original layout, images, hyperlinks, and styling remain intact while the textual content is replaced with its translated equivalent.

The feature is designed to improve accessibility and usability for users who prefer reading blog content in their native language. By supporting multiple translation providers and languages, the system offers flexibility and reliability in delivering translated content.

Several important requirements guide the implementation of this feature:

- Support for multiple translation providers.
- Support for multiple Indian languages.
- Ability to switch translation providers dynamically at runtime.
- Storage of user language preferences using cookies, enabling automatic translation when users revisit the page.
- Translation limited to content present during the initial page load, excluding editor or administrative pages.

6.2 Supported Languages

The Web Translation feature currently supports several major Indian languages in addition to English.

Language	**Code**
English	en-IN
Hindi	hi-IN
Tamil	ta-IN
Telugu	te-IN
Bengali	bn-IN
Kannada	kn-IN

These languages were selected to provide coverage across multiple linguistic regions of India while ensuring compatibility with the available translation providers.

6.3 Translation Workflow

The translation process follows a structured workflow that dynamically converts webpage text into the selected language.

1. The user selects a target language from the translation toolbar.
2. The translation system identifies all text nodes within the page's `DOM`.
3. Each text node is processed individually.
4. The system checks whether a translation already exists in the translation cache.
5. If a cached translation exists, it is returned immediately.

6. If the translation is not available in the cache, the system sends a request to the selected translation provider.

7. The translated text is returned and stored in the cache.

8. The translated text replaces the original content in the `DOM`.

9. After processing all text nodes, the entire webpage appears in the selected language without requiring a page refresh.

6.4 Translation Providers

The system supports multiple translation providers to improve flexibility and reliability.

- **Sarvam AI**

  The primary translation provider is Sarvam AI, which provides high-quality translations for Indian languages. Since the Sarvam AI SDK cannot be accessed directly from the browser, a Node.js proxy server is used to securely communicate with the API.

- **MyMemory Translation API**

  A secondary translation provider is the MyMemory REST API, which provides free translation services through HTTP requests. This provider acts as an alternative option and ensures that translation functionality remains available even if the primary provider is unavailable.

By supporting multiple providers, the system ensures reliability and enables users to switch providers dynamically.

6.5 System Architecture

The architecture of the Web Translation Feature consists of multiple JavaScript modules that work together to translate webpage content.

Layer	**File**	**Role**
Strategy Interface	`TranslationStrategy.js`	Declares translation interface methods
Concrete Strategy	`SarvamStrategy.js`	Calls Sarvam AI through a Node.js proxy
Concrete Strategy	`FetchStrategy.js`	Calls the MyMemory REST translation API
Context	`TranslationContext.js`	Maintains the active translation strategy
Cache	`TranslationCache.js`	Implements caching using IndexedDB
DOM Utilities	`domUtils.js`	Extracts and replaces page text nodes
Orchestrator	`translation.js`	Builds the translation toolbar and manages user interactio
Node Proxy	`frontend/server.js`	Express server proxy for Sarvam AI API
JSP Include	`tiles/head.jsp`	Loads translation scripts for every webpage

This modular architecture ensures that translation logic, caching, provider communication, and user interaction remain separated.

6.6 Design Patterns Used

The implementation of the Web Translation Feature uses several software design patterns to ensure flexibility and maintainability.

Strategy Pattern

The Strategy Pattern encapsulates different translation algorithms behind a common interface called `TranslationStrategy`. Each translation provider implements this interface, allowing the system to switch providers dynamically.

Instead of embedding provider-specific logic directly into the application, the system delegates translation requests to the currently selected strategy.

Advantages of this approach include:

- Simplified provider switching at runtime
- Independent testing of each provider implementation
- Easy addition of new translation providers
- Elimination of complex conditional logic

Roles in the Strategy Pattern:

GoF Role	Class
Strategy	`TranslationStrategy`
Concrete Strategy	`SarvamStrategy`
Concrete Strategy	`FetchStrategy`
Context	`TranslationContext`
Client	`translation.js`

6.7 Scalability and Performance Considerations

Several design decisions were implemented to ensure scalability and efficient performance.

1. **Caching Mechanism**

   The system uses IndexedDB caching through the `TranslationCache` module. Each translated text is stored along with a key consisting of:

   - URL
   - Target language
   - Translation provider
   - Content hash

   This cache-aside strategy ensures that repeated translations of the same content are avoided.

2. **Efficient DOM Processing**

   The system processes only text nodes within the page, avoiding unnecessary operations on HTML elements such as images or links.

3. **Runtime Provider Switching**

   Since translation providers are encapsulated through the Strategy Pattern, switching providers does not require changes to the core translation workflow.

These optimizations help reduce API calls, improve page responsiveness, and enhance the overall user experience.

6.8 User Interface Demonstration

The translation interface is integrated directly into the webpage through a translation toolbar.

The toolbar allows users to:

- Select their preferred target language
- Choose the translation provider
- Trigger translation for the current page

Once translation is activated, the page content updates dynamically. The translated text replaces the original text nodes while preserving the page layout and styling.

User preferences such as language and provider are stored using cookies, allowing the system to automatically apply translations when the user revisits the page.

6.9 Future Improvements

Several improvements can be considered to enhance the Web Translation Feature in future versions.

Possible improvements include:

- Adding support for additional Indian and international languages.
- Integrating additional translation providers for improved translation accuracy.
- Implementing batch translation requests to reduce API latency.
- Supporting translation of dynamic content loaded after page initialization.
- Providing user feedback mechanisms for correcting translation errors.

These improvements would further increase the accessibility and usability of the platform for multilingual audiences.

7 Task 5B: Translation Caching Feature

7.1 Feature Overview & Task Flow

The **Translation Caching Feature** improves the efficiency of the Web Translation system by reducing repeated API requests to external translation providers such as Sarvam AI. Instead of requesting translations for the same content every time a page is loaded, the system stores successful translation results locally using IndexedDB in the browser.

When a user revisits a page, previously translated text can be retrieved directly from the cache. This eliminates unnecessary API calls and significantly improves translation speed.

The caching mechanism follows a structured workflow:

1. When a page is translated, each text node is processed individually.
2. A cache key is generated for each text node using the following components:
    - Page URL
    - Target language
    - Translation provider
    - SHA-256 hash of the source text
3. The system checks whether a translation already exists in the cache.
4. If a cached translation exists, the stored value is returned immediately.
5. If the translation is not found in the cache, the system sends a request to the selected translation provider.
6. The translated text is returned and stored in the cache for future use.

When a user revisits the same page, the system recomputes the hash of each text node. If the hash matches the stored hash, the cached translation is reused and no API call is required.

If the source text changes, only the modified text node is retranslated while unchanged nodes continue to use cached translations. This selective update mechanism significantly improves performance.

Additionally, the system stores the user's preferred language and translation provider in cookies, allowing automatic translation of previously visited pages using cached results.

7.2 Design Principles Integrated

Several software design principles were applied to ensure that the translation caching system remains modular, maintainable, and extensible.

1. **Separation of Concerns**

The caching mechanism is implemented independently from the translation logic. Translation providers focus solely on generating translated text, while the cache layer handles storage and retrieval of translations.

2. **Single Responsibility Principle**

   Each component of the caching system performs a single well-defined task:

   - `TranslationCache` handles storage and retrieval of cached translations.
   - `CacheEventEmitter` manages event notifications.
   - `CacheLogger` handles logging and debugging of cache events.
   - `TranslationContext` coordinates translation requests and integrates caching logic.

   This separation ensures that modifications to one component do not affect the functionality of others.

3. **Extensibility**

   The caching system allows additional components such as analytics modules or monitoring tools to observe cache activity without modifying the core cache implementation.

4. **Cache-Aside Strategy**

   The system uses a cache-aside pattern, where the application first checks the cache before querying the translation provider. If the translation is not present, the system retrieves the translation from the provider and then stores it in the cache.

7.3 Leveraged Design Patterns

The implementation of the Translation Caching Feature uses the **Observer Pattern** to manage cache events.

Observer Pattern

The Observer Pattern is a behavioural design pattern that defines a one-to-many relationship between objects. When the state of a subject changes, all registered observers are automatically notified.

In this system:

- The `TranslationCache` component acts as the source of cache events.
- The `CacheEventEmitter` manages event registration and notification.
- Observers such as `CacheLogger` listen for cache events and react accordingly.

This design ensures that the cache system remains independent of logging, analytics, or debugging components.

Roles in the Observer Pattern

GoF Role	Class
Subject Interface	`CacheEventEmitter`
Concrete Observer	`CacheLogger`
Observable / Notifier	`TranslationCache`

Cache Events

The caching system emits several events to inform observers about cache activity.

Event	Description
`hit`	A cached translation was found and returned
`miss`	No cached translation exists; the provider will be called
`put`	A new translation result was stored in the cache
`error`	An IndexedDB operation failed
`invalidated`	Cache entries were removed (future functionality)

Because observers subscribe to events through the event emitter, new observers can be added without modifying the cache system.

7.4 Quality Attributes & Trade-offs

The Translation Caching Feature introduces several improvements in system performance and scalability while also involving certain trade-offs.

Performance Improvements

- **Caching**
  Caching significantly reduces the number of API requests sent to translation providers. When users revisit previously translated pages, translations can be retrieved instantly from the local cache, resulting in faster page translation.
- **Reduced Network Overhead**
  By avoiding redundant API calls, the system reduces network usage and decreases dependency on external translation services.
- **Scalability**
  Since caching occurs locally within the browser using IndexedDB, the system scales naturally with the number of users. Each user's browser maintains its own cache, preventing server-side bottlenecks.
- **Selective Retranslation**
  The use of SHA-256 hashing ensures that only modified text nodes are retranslated. If the hash of the text remains unchanged, the cached translation is reused.

Trade-offs

- Cached translations consume local browser storage.

- A complete page retranslation may occur if all text nodes change, although this is a rare scenario.
- Cache management policies may be required in the future to handle long-term storage limits.

Overall, the caching mechanism improves system efficiency while maintaining flexibility and scalability.

8 Task 6A: Community Pulse Dashboard

8.1 Introduction

The Community Pulse Dashboard is a feature designed to provide bloggers with insights into the engagement and discussion activity on their blog posts. It analyzes comments associated with a blog entry and summarizes the overall interaction patterns in a simple dashboard format. The dashboard processes comment data to generate key metrics such as the total number of comments, recent activity levels, sentiment classification, and frequently used keywords. These insights help authors quickly understand how readers are interacting with their content and whether discussions are positive, critical, or inquisitive. The system performs automated analysis on the comments using keyword matching and text processing techniques. The results are then displayed through a visually organized interface, enabling users to interpret discussion trends without manually reviewing each comment. This feature integrates backend analytics with a frontend dashboard to create a real-time overview of community engagement for individual blog entries.

8.2 Overview of Community Pulse Dashboard

The Community Pulse Dashboard provides a consolidated view of comment activity for a particular blog entry. It allows users to monitor reader engagement by displaying multiple discussion metrics in an organized format. The dashboard includes several key components:

1. **Comment Activity Metrics**

   The system calculates and displays metrics such as:

   - Total number of comments on the blog entry
   - Number of comments posted within the last seven days
   - Average number of comments per day
   - Overall activity level (HOT, ACTIVE, or QUIET)

   These metrics allow authors to quickly understand how actively their content is being discussed.

2. **Sentiment Analysis**

   The system categorizes comments into three types based on simple keyword detection:

- **Questions** – comments containing inquiry indicators such as "?"
- **Praise** – comments containing positive keywords like *excellent, amazing, great*
- **Critique** – comments containing critical keywords such as *problem, issue, concern*

This classification helps bloggers identify how readers are reacting to their content.

3. **Keyword Extraction**

   The dashboard extracts frequently occurring keywords from the comments after removing common stop words. These keywords highlight the main topics readers are discussing.

4. **User Interface Dashboard**

   The metrics and analysis results are presented in a visually structured interface consisting of:

   - Metrics summary tables
   - Sentiment breakdown tables
   - Keyword tags displaying important discussion topics

   This allows users to easily interpret engagement patterns without analyzing raw comment data.

8.3 Implementation Details

The implementation of the Community Pulse Dashboard follows a layered architecture and integrates multiple components across the backend and frontend of the system.

Backend Components

1. **DiscussionOverview.java**

   This class acts as a data container (POJO) that stores all computed metrics related to a blog entry's discussion.

   It includes attributes such as:

   - totalComments
   - commentsInLastSevenDays
   - averageCommentsPerDay
   - activityLevel
   - questionsCount
   - praiseCount
   - critiqueCount
   - topKeywords

   This object is returned after the comment analysis is completed.

2. **DiscussionOverviewCalculator.java**

   This class contains the core business logic for calculating the community pulse metrics.

   Its responsibilities include:

   - Fetching all comments associated with a blog entry
   - Performing sentiment detection on each comment
   - Extracting keywords from comment text
   - Calculating activity levels and averages

   The algorithm performs a single-pass analysis over all comments, giving it a time complexity of approximately:

   $$O(n \times m)$$

   where:

   - n = number of comments
   - m = average comment length

3. **Unit Testing**

   The system includes unit tests implemented in:

   `DiscussionOverviewCalculatorTest.java`

   Two main test cases are included:

   - Analysis with multiple comments to verify correct metric calculation
   - Handling blog entries with no comments to ensure null-safe behavior

   These tests ensure the reliability of the comment analysis logic.

Frontend Components

1. **PulseView.java**

   This class acts as a Struts2 Action Controller that manages the request flow.

   Its responsibilities include:

   - Receiving the blog entry ID from the request
   - Validating the request parameters
   - Verifying that the logged-in user owns the blog entry
   - Invoking the DiscussionOverviewCalculator
   - Passing the calculated data to the view layer

2. **PulseView.jsp**

   This JSP file implements the dashboard interface. It displays:

   - Key metrics summary tables
   - Sentiment analysis breakdown
   - Extracted keyword tags
   - Navigation options for the user

   The UI uses Struts2 tags, responsive styling, and visual indicators such as icons and badges.

Configuration

The feature also required updates to configuration files including:

- `struts.xml` – to define the action mapping for the pulse dashboard
- `tiles.xml` – to integrate the new page layout
- `ApplicationResources.properties` – to add internationalized text labels for the UI

These configurations ensure proper integration of the feature within the existing Apache Roller application framework.within the existing Apache Roller application framework.

8.4 Design Patterns Used

Several design patterns were used to ensure maintainability, modularity, and separation of concerns in the Community Pulse Dashboard.

1. **Facade Pattern**

   The `DiscussionOverviewCalculator` acts as a facade that simplifies the interaction between different subsystems.

   Instead of the controller directly performing complex comment analysis operations, it simply calls the calculator. The calculator internally handles tasks such as sentiment detection, keyword extraction, and metric computation.

   This reduces coupling and simplifies the controller logic.

2. **Model–View–Controller (MVC)**

   The system follows the MVC architecture:

   - **Model**: DiscussionOverview – stores discussion metrics
   - **View**: PulseView.jsp – displays the dashboard interface
   - **Controller**: PulseView.java – handles request processing and coordinates between model and view

This separation improves maintainability and scalability of the application.

3. **Strategy Pattern**

   Sentiment detection is implemented using multiple detection strategies based on keyword patterns. Each comment is analyzed using predefined rules to classify it into questions, praise, or critique categories.

   This pattern allows the sentiment detection logic to be easily extended with additional strategies in the future.

4. **Template Method Pattern**

   The Struts2 framework provides a predefined lifecycle for action execution. The application follows this template when processing requests, where the `execute()` method follows the framework's structured request handling flow.

5. **Dependency Injection**

   Dependencies such as `DiscussionOverviewCalculator` and `WeblogEntryManager` are injected into the controller using the framework's configuration.

   This reduces tight coupling between components and improves testability.

6. **Single Responsibility Principle**

   Each class in the system has a clearly defined responsibility:

   - `PulseView.java` → request handling and security checks
   - `DiscussionOverviewCalculator.java` → business logic
   - `DiscussionOverview.java` → data storage
   - `PulseView.jsp` → user interface rendering

   This design improves code readability, modularity, and maintainability.

8.5 Conclusion

The Community Pulse Dashboard provides a powerful yet simple mechanism for analyzing reader engagement in blog discussions. By automatically processing comment data, the feature transforms raw text into meaningful insights that help bloggers understand community reactions to their content. The implementation combines backend analytics, frontend visualization, and secure request handling to create a cohesive full-stack feature. The use of design patterns such as MVC, Facade, and Strategy improves code organization and maintainability, while the single-pass analysis algorithm ensures efficient processing even when many comments are present. Overall, the Community Pulse Dashboard enhances the usability of the blogging platform by giving authors a quick overview of discussion trends, sentiment distribution, and important keywords. This enables bloggers to better understand audience feedback and encourages more data-driven content management.

9 Task 6B: Community Pulse Conversation Breakdown

9.1 Overview of Task

Task 6B extends the functionality of the Community Pulse Dashboard by introducing a deeper analysis of discussions occurring within blog comments. While Task 6A focuses on lightweight statistical indicators such as activity levels and response types, Task 6B performs a more advanced conversation breakdown analysis that identifies key themes and generates a structured summary of reader discussions. The system analyzes comments using two different approaches: classical Natural Language Processing (NLP) techniques and AI-assisted large language model (LLM) analysis. Depending on system availability and configuration, the system dynamically selects the appropriate method for generating insights. The goal of this task is to provide blog authors with meaningful summaries of reader conversations, helping them quickly understand recurring topics, major concerns, and important feedback without manually reading all comments. The implementation introduces a modular architecture using multiple design patterns such as Strategy, Factory, and Template Method patterns to ensure extensibility and maintainability of the analysis engine.

9.2 SOLID Design Principles Applied

The implementation of the Conversation Breakdown feature follows the SOLID principles to improve modularity, maintainability, and extensibility.

9.2.1 Single Responsibility Principle

The Single Responsibility Principle states that each class should have only one reason to change. In this implementation, responsibilities are clearly separated across multiple classes:

- `ConversationBreakdownCalculator` manages the coordination and selection of analysis strategies.
- `InsightGenerationStrategy` defines the contract for generating insights from comments.
- `KeywordClusteringStrategy` performs classical keyword-based analysis.
- `LLMSummarizationStrategy` performs AI-based summarization using an external API.
- `GroqAPIClient` handles communication with the AI service.

By isolating each responsibility, the system becomes easier to maintain and modify without affecting other components.

9.2.2 Open/Closed Principle

The Open/Closed Principle states that software components should be open for extension but closed for modification.

The system achieves this by defining the `InsightGenerationStrategy` interface, which allows new analysis strategies to be added without modifying existing code. For example, developers can introduce additional AI models or alternative NLP algorithms by simply implementing this interface.

This approach enables the system to evolve with minimal changes to the existing architecture.

9.2.3 Liskov Substitution Principle

The Liskov Substitution Principle ensures that subclasses can replace their base class without affecting system behavior.

All strategy implementations such as:

- `KeywordClusteringStrategy`
- `LLMSummarizationStrategy`

adhere to the `InsightGenerationStrategy` interface. This allows the `ConversationBreakdownCalculator` to use any strategy interchangeably without needing to know the implementation details.

As a result, the system can switch between classical and AI-based analysis seamlessly.

9.2.4 Interface Segregation Principle

The Interface Segregation Principle suggests that interfaces should be specific and minimal rather than large and general.

The `InsightGenerationStrategy` interface defines only the essential methods required for generating insights from comments. This prevents classes from implementing unnecessary functionality and keeps the design clean and focused.

By keeping the interface small and specialized, each analysis strategy implements only the functionality it actually requires.

9.2.5 Dependency Inversion Principle

The Dependency Inversion Principle states that high-level modules should depend on abstractions rather than concrete implementations.

In this architecture, the `ConversationBreakdownCalculator` depends on the abstraction defined by the `InsightGenerationStrategy` interface rather than concrete strategy classes.

This allows different strategies to be injected or selected dynamically at runtime, improving flexibility and decoupling system components.

9.3 Design Patterns Used

Several design patterns were implemented to support modular analysis and flexible strategy selection.

9.3.1 Strategy Pattern

The Strategy Pattern allows the system to switch between different analysis methods at runtime.

In this implementation, the `InsightGenerationStrategy` interface defines the contract for generating insights. Multiple strategies implement this interface:

- `KeywordClusteringStrategy` (classical NLP)
- `LLMSummarizationStrategy` (AI-based analysis)

The system selects the appropriate strategy depending on the environment and configuration. This design enables easy addition of new analysis methods without modifying existing logic.

9.3.2 Factory Pattern

The Factory Pattern is used to create and manage the appropriate strategy objects.

The `ConversationBreakdownCalculator` acts as a factory and coordinator that determines which strategy should be used based on runtime conditions such as API availability.

By centralizing the strategy creation process, the system simplifies management of different analysis methods and improves maintainability.

9.3.3 Template Pipeline Approach

A Template Method–like pipeline structure is used to enforce a consistent workflow for conversation analysis.

Each analysis strategy follows a similar sequence of steps:

1. Collect comments for the blog entry
2. Preprocess the comment text
3. Perform theme detection or clustering
4. Identify representative comments
5. Generate a structured conversation breakdown

This template-based approach ensures consistency across different analysis strategies while allowing each strategy to implement its own specific logic.

9.4 Implementation Architecture

The implementation of the Conversation Breakdown feature consists of multiple backend and frontend components that work together to generate and display discussion insights.

9.4.1 Core Files Added

Several new classes were introduced to support the conversation breakdown feature.

1. **Analysis Engine**
   - `InsightGenerationStrategy.java` – Defines the interface for different analysis strategies.
   - `ConversationBreakdown.java` – Data model storing themes and summaries generated from analysis.
   - `Theme.java` – Represents a discussion theme with representative comments.
   - `KeywordFrequency.java` – Utility class used to track keyword occurrence frequency.
2. **Analysis Implementations**
   - `KeywordClusteringStrategy.java` – Performs classical keyword clustering using TF-IDF style analysis.
   - `LLMSummarizationStrategy.java` – Uses AI models to extract themes and summaries.
   - `ConversationBreakdownCalculator.java` – Coordinates strategy selection and execution.
3. **AI Integration**
   - `GroqAPIClient.java` – Handles communication with the Groq API used for AI-based comment analysis.
4. **User Interface**
   - `PulseBreakdown.java` – Controller for the conversation breakdown view.
   - `ConversationBreakdownView.jsp` – Main UI for displaying themes and summaries.
   - `ConversationBreakdownViewSimple.jsp` – Simplified interface for fallback scenarios.

9.4.2 Existing Files Updated

Some existing files were updated to integrate the new feature with the existing dashboard.

- `tiles.xml` – Updated to register the new conversation breakdown view.
- `Entries.jsp` – Updated to include a "View Pulse" button allowing users to access the analysis dashboard.

These updates allow the new functionality to be seamlessly integrated with the existing Apache Roller user interface.

9.5 Method Behaviors

The conversation breakdown system supports multiple analysis approaches.

9.5.1 Classical NLP

The classical method uses keyword-based statistical techniques to analyze comment text.

This approach performs the following steps:

1. Extract keywords from comments
2. Calculate keyword frequency
3. Cluster comments based on common keywords
4. Identify recurring discussion points

This method is computationally efficient and does not require external AI services.

9.5.2 LLM-Assisted

The LLM-assisted approach uses an external AI model via the Groq API to analyze comments.

The AI model performs more advanced analysis including:

- Identifying discussion themes
- Selecting representative comments for each theme
- Generating a concise summary of the overall conversation

This approach produces more meaningful and human-like summaries compared to classical statistical methods.

9.5.3 Hybrid Approach

The system implements a hybrid design that combines classical and AI-based approaches.

- If the AI service is available and properly configured, the system performs LLM-based analysis.
- Otherwise, it automatically falls back to the classical keyword clustering approach.

This hybrid architecture ensures both advanced analysis capability and system reliability.

9.6 Auto-Selection Logic

The system automatically selects the most suitable analysis method based on runtime conditions.

The selection process follows these steps:

1. When a user requests the conversation breakdown, the system checks whether an AI API key is available.
2. If the AI service is available, the `LLMSummarizationStrategy` is selected.
3. If the AI service is unavailable or fails, the system automatically switches to `KeywordClusteringStrategy`

This fallback mechanism ensures that the feature always provides useful results even when external AI services cannot be accessed.

9.7 UI and Interaction

The conversation breakdown feature includes a user interface designed to clearly present discussion insights. The UI consists of:

- Theme sections showing major discussion topics
- Representative comments illustrating each theme
- Overall conversation summary generated by the analysis system
- Navigation buttons allowing users to return to the main dashboard

The interface is implemented using JSP pages with Bootstrap-based styling to ensure a responsive and user-friendly experience. Users can access this feature directly from the Community Pulse Dashboard by selecting the "View Conversation Breakdown" option.

10 Team Member Contributions

Roll Number	Name	Contribution
2025204033	Deepti Koranga	Task 1A (Stars Feature), Task 1B (Trending Blogs), Task 6A (Community Pulse Dashboard)
2025204012	VSS Bharadwaja	Task 2 (Transforming Feeds), Task 6B (Conversation Breakdown), BONUS
2025204037	Rahul Ebenezer Chand	Task 3 (Bug Reporting System), BONUS
2025204002	Sanapathi Kishore Naidu	Task 4 (Admin Dashboard) and Task 6A (Community Pulse Dashboard)
2025204018	K. Anil Kumar	Task 5A (Web Translation), Task 5B (Translation Caching), Task 6A (Community Pulse Dashboard), BONUS